

Learning-Based Sketches: Learned Bloom Filters

Vyomika Vasireddy (23110363)

vyomika.vasireddy@iitgn.ac.in

Indian Institute of Technology Gandhinagar
Gandhinagar, Gujarat, India

Boinapally Dhanvini (24110079)

boinapally.dhanvini@iitgn.ac.in

Indian Institute of Technology Gandhinagar
Gandhinagar, Gujarat, India

Bhavana Ravi (23110274)

bhavana.ravi@iitgn.ac.in

Indian Institute of Technology Gandhinagar
Gandhinagar, Gujarat, India

Snehitha Bollineni (24110081)

snehitha.bollineni@iitgn.ac.in

Indian Institute of Technology Gandhinagar
Gandhinagar, Gujarat, India

Abstract

This project studies learning-based sketches with a focus on learned Bloom filters. Traditional Bloom filters are data-oblivious and do not adapt to dataset properties. Learned Bloom filters incorporate machine learning classifiers to exploit patterns in the data and improve performance. By combining a learned model with a backup Bloom filter, we ensure **zero false negatives** as a strict correctness guarantee while reducing false positives under equal memory constraints. This approach is especially useful in large-scale systems such as databases and caching, where efficient membership queries are critical. We compare learned Bloom filters with standard Bloom filters using metrics such as false positive rate, memory usage, and query time, and study how data distribution and model choice affect performance.

1 Introduction and Bloom Filter Basics

Bloom filters are data structures that save space and are utilised for quick membership queries. To determine whether an element is present in a set, they employ several mappings and a bit array. They never generate false negatives, but they could generate false positives.

The fact that Bloom filters are data-oblivious, which means they treat all inputs equally and do not exploit data patterns, is a significant drawback. Unnecessary false positives result from this. To overcome this constraint, we developed a learned Bloom filter that applies it after identifying patterns in the data. We also investigated the expanded version of the sandwich Bloom filter. Overlaps and false positives may result from mapping elements to multiple locations in a fixed-size array in a typical Bloom filter.

Implementation

The system uses a two-stage decision-making method. First, the structure of the incoming elements, such as their length and character composition, is examined.

Every element is given a confidence value. The element is immediately accepted when trust is high. If not, a backup filtering stage receives it.

By ensuring that no legitimate element is overlooked, this fallback stage maintains zero false negatives. By using this method, the system becomes data-aware instead of data-oblivious. By using the fallback mechanism only when needed, this procedure enables

the system to handle confident scenarios swiftly. Through the fallback method, this approach minimises needless computations while maintaining accuracy.

Dataset Design

To understand the system’s behaviour in both controlled and real-world scenarios, we assessed it using both synthetic and real datasets.

A variety of data distributions were intended to be represented using synthetic datasets. All of the elements in uniform data are random and lack patterns. Zipf data are skewed, with some elements occurring more frequently than others. Inherent patterns can be observed in the composition of structured information, such as words and URLs.

Practical performance was evaluated using real datasets. Real-world datasets and URL datasets, which add inherent variety and complexity, are among them. We may analyse the system’s performance under both ideal and realistic conditions using both types of datasets.

2 Optimisation of Bloom Filter

By adding a pattern-based judgment step before filtering, we improved the conventional Bloom filter. This reduces the needless checks by telling the system to recognise and accept components that directly match learned patterns.

“Threshold value”, which controls the system’s level of trust in pattern-based choice, is an important factor here. A higher threshold moves the choice toward the fallback filtering stage, while a lower threshold increases reliance on the learned component.

This optimisation improves memory efficiency and reduces false positives, particularly in datasets with strong patterns. By filtering multiple queries early, it also reduces strain on the Bloom filter, increasing overall system efficiency.

3 Results and Comparisons for Different Datasets

We compare the performance of different approaches using false positive rate, memory usage, and query time. The results are evaluated across both synthetic and real datasets to understand how the system behaves under different conditions.

3.1 False Positive Rate Analysis

Figure 1 shows that the trained Bloom filter exhibits no improvement on uniform datasets, both real and simulated. We obtain a

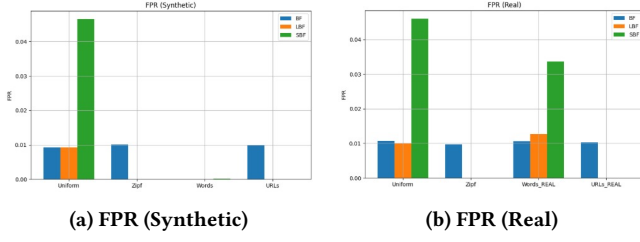


Figure 1: False positive rate comparison on synthetic and real datasets.

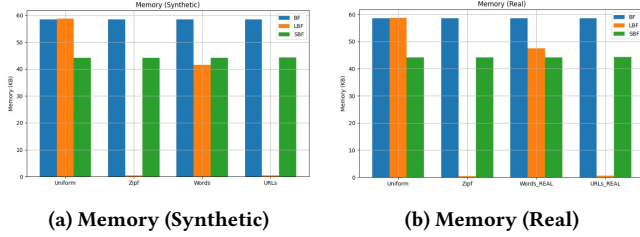


Figure 2: Memory usage comparison on synthetic and real datasets.

false-positive rate of **0.009–0.010**, comparable to that of a typical Bloom filter. The false-positive rate rises to roughly **0.046–0.047** when using the sandwich approach, which performs poorly.

In the case of Zipf datasets, the learnt Bloom filter reduces the false-positive rate (FPR) from around **0.009–0.010** to **0.0000**, demonstrating strong performance when patterns are present. In this instance, the sandwich approach also achieves a **nearly 0% false-positive rate**.

Synthetic data produces almost no false positives on word datasets using any strategy. On actual data, however, the learned Bloom filter slightly increases the false-positive rate from roughly **0.0106** to **0.0712**, and the sandwich approach further increases it to roughly **0.033–0.034**.

For URL datasets, both synthetic and real data show great improvement. The learned Bloom filter reduces the false positive rate from about **0.010** to **0.0000**, achieving perfect performance.

These results show that performance depends strongly on the presence of patterns in the data, with significant improvements observed only in structured datasets such as URLs.

3.2 Analysis of Memory for Different Datasets

The standard Bloom filter uses approximately **58–59 KB** of memory across all datasets.

On uniform datasets, the learned Bloom filter also uses around **58–59 KB**, showing no improvement due to the absence of patterns. The sandwich Bloom filter reduces memory usage to approximately **44 KB**, but this does not improve accuracy too much.

Trained Bloom filter maintains a FPR of **0.0000** while reducing memory on Zipf datasets from nearly **58 KB** to **~0.4 KB**. This suggests that when strong patterns are detected, trained approaches can serve as a substitute for massive storage.

The learned Bloom filter reduces memory usage for word datasets to **41–47 KB**, while the regular Bloom filter requires **58 KB**. There

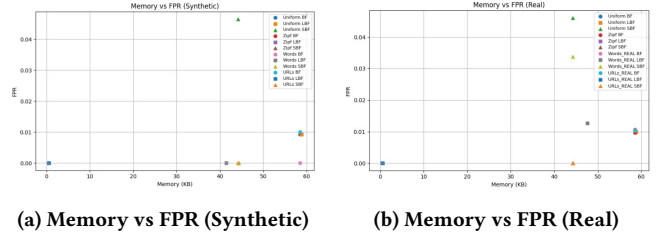


Figure 3: Memory versus false positive rate trade-off.

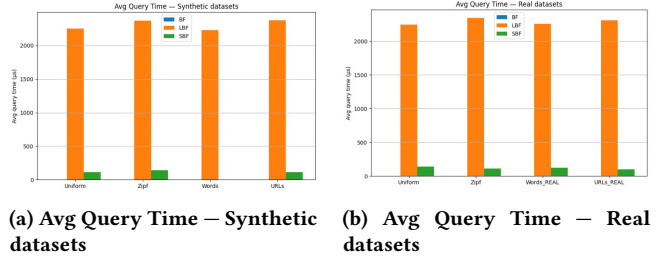


Figure 4: Average query time comparison across datasets.

is just a slight improvement using the sandwich method, which requires about **44 KB**.

The trained Bloom filter once again demonstrates notable improvement on URL datasets, achieving a false-positive rate of **0.0000** while reducing memory from approximately **58 KB** to **approximately 0.5 KB**.

These results show that learned Bloom filters are highly effective at reducing memory when strong patterns exist, but provide little to no benefit for uniform data. So, memory efficiency is linked to how well patterns can be learned from the data.

3.3 Memory and Accuracy Trade-off

The standard Bloom filter uses about **58–59 KB** of memory with a false-positive rate of about **0.01**.

The learned Bloom filter achieves the best trade-off on structured datasets, using only **0.4–0.5 KB** of memory with a false-positive rate of **0.0000** on the Zipf and URL datasets.

For word datasets, it uses about **41–47 KB** of memory and has a slightly higher false-positive rate of **0.012–0.013**.

The Bloom filter for the sandwich uses around **44 KB** of memory but may increase the false-positive rate to **0.03–0.046**; it will be less efficient (not as good).

3.4 Analysis of Query Time for Different Datasets

The standard Bloom filter is the fastest; from the graph above, we can see that it takes **1.1 to 1.4 microseconds** per query. The learned Bloom filter is significantly slower, taking about **2200–2400 microseconds** across all datasets.

The sandwich Bloom filter reduces this to about **100–150 microseconds** on most datasets. In some cases, such as synthetic

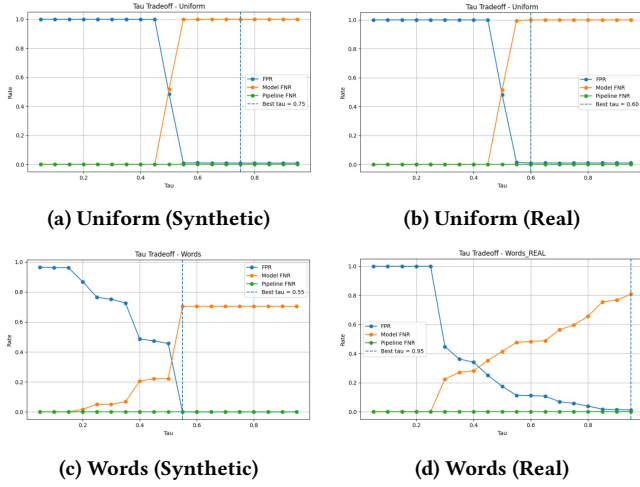


Figure 5: Effect of threshold on system performance.

word datasets, it can be low (~ 1.2 microseconds), close to that of a standard Bloom filter.

These results show that learned Bloom filters improve memory and accuracy, but at the cost of significantly higher query time.

3.5 Threshold Analysis

The threshold value controls how much the system relies on the learned component versus the fallback filtering stage.

For uniform datasets, the optimal threshold is around **0.60 (real)** and **0.75 (synthetic)**, since the model is unable to learn useful patterns and must rely more on the fallback stage.

For word datasets, the optimal threshold is around **0.95 (real)** and **0.55 (synthetic)**, reflecting the higher complexity and variability in real-world data.

Across all datasets, increasing the threshold reduces false positives but increases reliance on fallback filtering, while lower thresholds rely more on the learned model.

This shows that optimal threshold selection depends strongly on data complexity and the structure.

4 Findings Regarding the Sandwich Bloom Filter

After adding a filtering step before the learned model, the Sandwich Bloom filter prevents unnecessary queries early on. From a conceptual standpoint, this design creates a multi-layered strategy to increase efficiency by combining quick initial filtering with learned decision-making skills.

From the results, we can see and tell that the accuracy is guaranteed to be higher because the sandwich Bloom filter continuously maintains zero false negatives across all datasets. Also, it uses **44 KB** less memory than the standard Bloom filter, which uses roughly **58 KB**, making it more memory-efficient. The method achieves very low false-positive rates of **0.0000** on structured datasets such as Zipf and URLs, demonstrating how well the combined filtering approach effectively leverages data patterns.

Even in fully complex datasets, sandwich design provides balanced performance by combining filtering and learning. We can also see the improved query times compared to the learned Bloom filter. This Scenario shows that multi-stage filtering can enhance efficiency and provide a flexible framework that balances memory, accuracy, and speed (Query time) across various datasets, such as URLs, words, and uniform datasets.

5 Conclusion

Throughout the project, we have seen and can attest that adding adaptive learning to traditional data structures, such as bloom filters, can strongly improve the performance by leveraging data patterns. Instead of treating all inputs uniformly, the system adapts to data characteristics, making it even more efficient at decision-making.

In structured datasets, the learned Bloom filter achieves substantial improvements, reducing memory usage from about **58 KB to less than 0.5 KB** and getting a **FPR of 0.0000**. The main thing is that learned methods can replace large storage requirements with compact, pattern-based representations. This method is applicable and adaptable to all other kinds of data. It gets high accuracy and memory efficiency on datasets with strong patterns, and the fallback mechanism ensures dependable performance for more complex data.

Also, compared to learned-only methods, the sandwich Bloom filter offers a balanced combination of memory efficiency, accuracy, and faster query times by introducing a multi-stage filtering process. To conclude, our experience and experiments show that trained Bloom filters offer a potent and flexible solution, particularly in situations when significant patterns are present. When properly adjusted, they provide a useful enhancement over conventional techniques while preserving accuracy and effectiveness.

Acknowledgments

We thank Dr. Anirban Dasgupta and the CS 328 teaching assistants for their guidance, feedback, and support throughout this project.

References

- [1] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. The case for learned index structures. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, pages 489–504, 2018.
- [2] Burton H. Bloom. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426, 1970.
- [3] Michael Mitzenmacher. A model for learned Bloom filters and optimizing by sandwiching. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.